Profiler-to-optimizer bottleneck playbook

A reference CSL benchmark can teach the agent where the cycles go, not just whether the run passed.

Profiler signal	Bottleneck insight	CSL action
sim.log stats	Classify stall-bound, fabric-heavy, or idle-tail behavior.	Unblock queues; reduce fabric traffic; rebalance PE work.
dispatch traces	Find idle PEs or data movement dominating arithmetic.	Use DSD chaining; fuse loops; reduce @fmovs.
backpressure	Localize congested PE, color, or halo direction.	Reorder send/recv; add async FIFO; split colors.
wavelet flow traces	Detect bursty long-tail communication latency.	Stagger phases; overlap receive with compute.
ELF / symbols	Expose SRAM pressure, large sections, or task bloat.	Shrink buffers; use comptime constants; repartition.
debug_util post-mortem	Spot wrong PE-local state after execution.	Inspect symbols; fix distribution/reduction; rerun.

Slide takeaway: cycles alone rank candidates; profiler context explains which architectural lever to try next.

Sources: PROFILING_ARCHITECTURE.md and HARDWARE_VALIDATION_20260706.md. Current implementation logs sim.log/profile context; deeper CTF parsing is the next step.